

AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation

Qingyun Wu[†], Gagan Bansal^{*}, Jieyu Zhang[±], Yiran Wu[†], Beibin Li^{*}

Erkang Zhu^{*}, Li Jiang^{*}, Xiaoyun Zhang^{*}, Shaokun Zhang[†], Jiale Liu[∓]

Ahmed Awadallah^{*}, Ryen W. White^{*}, Doug Burger^{*}, Chi Wang^{*1}

^{*}Microsoft Research, [†]Pennsylvania State University

[±]University of Washington, [∓]Xidian University

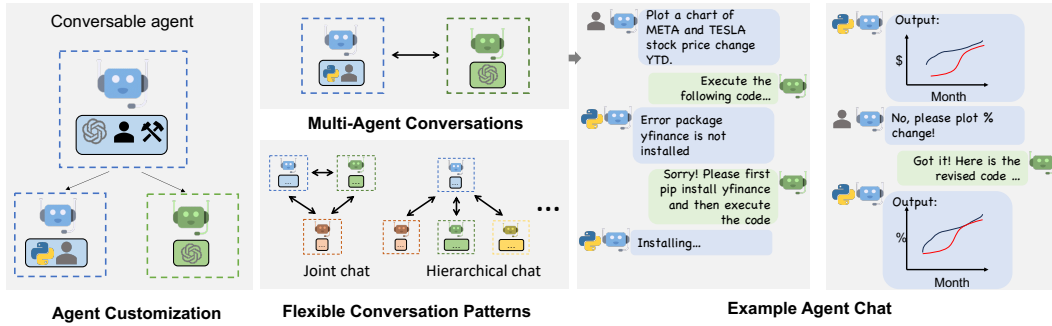


Figure 1: AutoGen enables diverse LLM-based applications using multi-agent conversations. (Left) AutoGen agents are conversable, customizable, and can be based on LLMs, tools, humans, or even a combination of them. (Top-middle) Agents can converse to solve tasks. (Right) They can form a chat, potentially with humans in the loop. (Bottom-middle) The framework supports flexible conversation patterns.

Abstract

AutoGen² is an open-source framework that allows developers to build LLM applications via multiple *agents* that can converse with each other to accomplish tasks. AutoGen agents are customizable, *conversable*, and can operate in various modes that employ combinations of LLMs, human inputs, and tools. Using AutoGen, developers can also flexibly define agent interaction behaviors. Both natural language and computer code can be used to program flexible conversation patterns for different applications. AutoGen serves as a generic framework for building diverse applications of various complexities and LLM capacities. Empirical studies demonstrate the effectiveness of the framework in many example applications, with domains ranging from mathematics, coding, question answering, operations research, online decision-making, entertainment, etc.

¹Corresponding author. Email: auto-gen@outlook.com

²<https://github.com/microsoft/autogen>

1 Introduction

Large language models (LLMs) are becoming a crucial building block in developing powerful *agents* that utilize LLMs for reasoning, tool usage, and adapting to new observations (Yao et al., 2022; Xi et al., 2023; Wang et al., 2023b) in many real-world tasks. Given the expanding tasks that could benefit from LLMs and the growing task complexity, an intuitive approach to scale up the power of agents is to use multiple agents that cooperate. Prior work suggests that multiple agents can help encourage divergent thinking (Liang et al., 2023), improve factuality and reasoning (Du et al., 2023), and provide validation (Wu et al., 2023). In light of the intuition and early evidence of promise, it is intriguing to ask the following question: *how* can we facilitate the development of LLM applications that could span a broad spectrum of domains and complexities based on the multi-agent approach?

Our insight is to use *multi-agent conversations* to achieve it. There are at least three reasons confirming its general feasibility and utility thanks to recent advances in LLMs: First, because chat-optimized LLMs (e.g., GPT-4) show the ability to incorporate feedback, LLM agents can cooperate through *conversations* with each other or human(s), e.g., a dialog where agents provide and seek reasoning, observations, critiques, and validation. Second, because a single LLM can exhibit a broad range of capabilities (especially when configured with the correct prompt and inference settings), conversations between differently configured agents can help combine these broad LLM capabilities in a modular and complementary manner. Third, LLMs have demonstrated ability to solve complex tasks when the tasks are broken into simpler subtasks. Multi-agent conversations can enable this partitioning and integration in an intuitive manner. How can we leverage the above insights and support different applications with the common requirement of coordinating multiple agents, potentially backed by LLMs, humans, or tools exhibiting different capacities? We desire a multi-agent conversation framework with generic abstraction and effective implementation that has the flexibility to satisfy different application needs. Achieving this requires addressing two critical questions: (1) How can we design individual agents that are capable, reusable, customizable, and effective in multi-agent collaboration? (2) How can we develop a straightforward, unified interface that can accommodate a wide range of agent conversation patterns? In practice, applications of varying complexities may need distinct sets of agents with specific capabilities, and may require different conversation patterns, such as single- or multi-turn dialogs, different human involvement modes, and static vs. dynamic conversation. Moreover, developers may prefer the flexibility to program agent interactions in natural language or code. Failing to adequately address these two questions would limit the framework’s scope of applicability and generality.

While there is contemporaneous exploration of multi-agent approaches,³ we present AutoGen, a generalized multi-agent conversation framework (Figure 1), based on the following new concepts.

- 1 **Customizable and conversable agents.** AutoGen uses a generic design of agents that can leverage LLMs, human inputs, tools, or a combination of them. The result is that developers can easily and quickly create agents with different roles (e.g., agents to write code, execute code, wire in human feedback, validate outputs, etc.) by selecting and configuring a subset of built-in capabilities. The agent’s backend can also be readily extended to allow more custom behaviors. To make these agents suitable for multi-agent conversation, every agent is made *conversable* – they can receive, react, and respond to messages. When configured properly, an agent can hold multiple turns of conversations with other agents autonomously or solicit human inputs at certain rounds, enabling human agency and automation. The conversable agent design leverages the strong capability of the most advanced LLMs in taking feedback and making progress via chat and also allows combining capabilities of LLMs in a modular fashion. (Section 2.1)
- 2 **Conversation programming.** A fundamental insight of AutoGen is to simplify and unify complex LLM application workflows as multi-agent conversations. So AutoGen adopts a programming paradigm centered around these inter-agent conversations. We refer to this paradigm as *conversation programming*, which streamlines the development of intricate applications via two primary steps: (1) defining a set of conversable agents with specific capabilities and roles (as described above); (2) programming the interaction behavior between agents via conversation-centric *computation* and *control*. Both steps can be achieved via a fusion of natural and programming languages to build applications with a wide range of conversation patterns and agent behaviors. AutoGen provides ready-to-use implementations and also allows easy extension and experimentation for both steps. (Section 2.2)

³We refer to Appendix A for a detailed discussion.

AutoGen also provides a collection of multi-agent applications created using conversable agents and conversation programming. These applications demonstrate how AutoGen can easily support applications of various complexities and LLMs of various capabilities. Moreover, we perform both evaluation on benchmarks and a pilot study of new applications. The results show that AutoGen can help achieve outstanding performance on many tasks, and enable innovative ways of using LLMs, while reducing development effort. (Section 3 and Appendix D)

2 The AutoGen Framework

To reduce the effort required for developers to create complex LLM applications across various domains, a core design principle of AutoGen is to streamline and consolidate multi-agent workflows using multi-agent conversations. This approach also aims to maximize the reusability of implemented agents. This section introduces the two key concepts of AutoGen: conversable agents and conversation programming.

2.1 Conversable Agents

In AutoGen, a *conversable agent* is an entity with a specific role that can pass messages to send and receive information to and from other conversable agents, e.g., to start or continue a conversation. It maintains its internal context based on sent and received messages and can be configured to possess a set of capabilities, e.g., enabled by LLMs, tools, or human input, etc. The agents can act according to programmed behavior patterns described next.

Agent capabilities powered by LLMs, humans, and tools. Since an agent’s capabilities directly influence how it processes and responds to messages, AutoGen allows flexibility to endow its agents with various capabilities. AutoGen supports many common composable capabilities for agents, including **1) LLMs.** LLM-backed agents exploit many capabilities of advanced LLMs such as role playing, implicit state inference and progress making conditioned on conversation history, providing feedback, adapting from feedback, and coding. These capabilities can be combined in different ways via novel prompting techniques⁴ to increase an agent’s skill and autonomy. AutoGen also offers enhanced LLM inference features such as result caching, error handling, message templating, etc., via an enhanced LLM inference layer. **2) Humans.** Human involvement is desired or even essential in many LLM applications. AutoGen lets a human participate in agent conversation via human-backed agents, which could solicit human inputs at certain rounds of a conversation depending on the agent configuration. The default *user proxy* agent allows *configurable* human involvement levels and patterns, e.g., frequency and conditions for requesting human input including the option for humans to skip providing input. **3) Tools.** Tool-backed agents have the capability to execute tools via code execution or function execution. For example, the default user proxy agent in AutoGen is able to execute code suggested by LLMs, or make LLM-suggested function calls.

Agent customization and cooperation. Based on application-specific needs, each agent can be configured to have a mix of basic back-end types to display complex behavior in multi-agent conversations. AutoGen allows easy creation of agents with specialized capabilities and roles by reusing or extending the built-in agents. The yellow-shaded area of Figure 2 provides a sketch of the built-in agents in AutoGen. The `ConversableAgent` class is the highest-level agent abstraction and, by default, can use LLMs, humans, and tools. The `AssistantAgent` and `UserProxyAgent` are two pre-configured `ConversableAgent` subclasses, each representing a common usage mode, i.e., acting as an AI assistant (backed by LLMs) and acting as a human proxy to solicit human input or execute code/function calls (backed by humans and/or tools).

In the example on the right-hand side of Figure 1, an LLM-backed assistant agent and a tool- and human-backed user proxy agent are deployed together to tackle a task. Here, the assistant agent generates a solution with the help of LLMs and passes the solution to the user proxy agent. Then, the user proxy agent solicits human inputs or executes the assistant’s code and passes the results as feedback back to the assistant.

⁴Appendix C presents an example of such novel prompting techniques which empowers the default LLM-backed assistant agent in AutoGen to converse with other agents in multi-step problem solving.

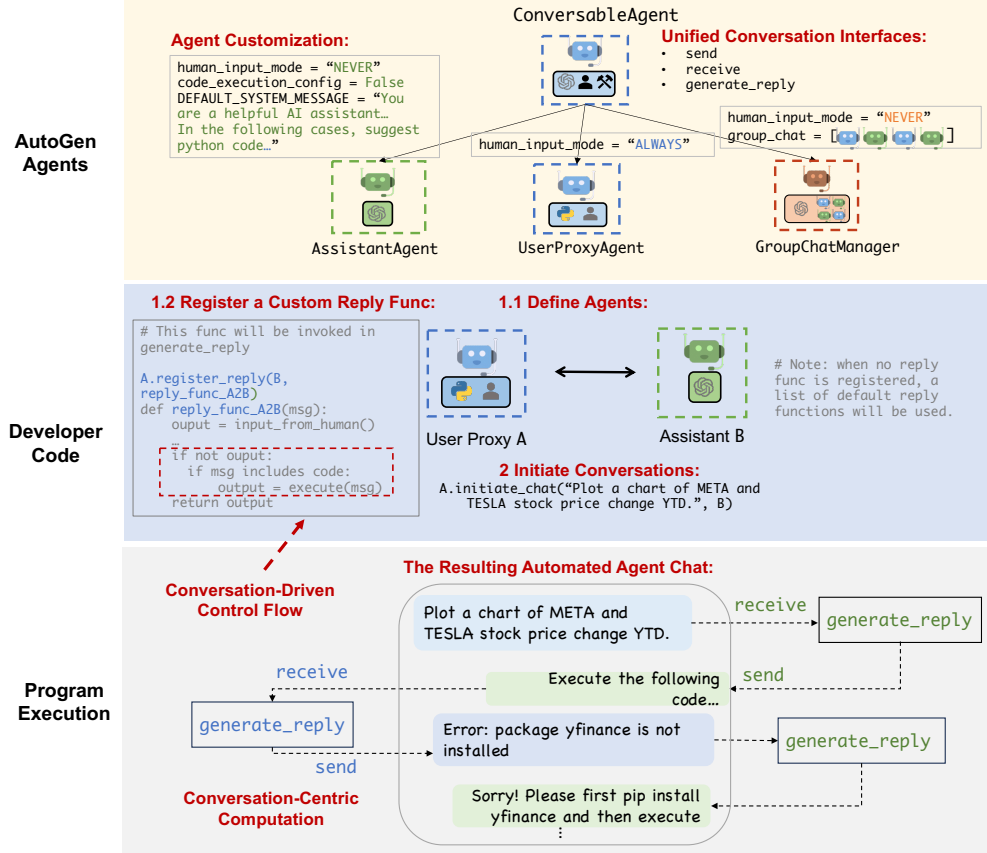


Figure 2: Illustration of how to use AutoGen to program a multi-agent conversation. The top sub-figure illustrates the built-in agents provided by AutoGen, which have unified conversation interfaces and can be customized. The middle sub-figure shows an example of using AutoGen to develop a two-agent system with a custom reply function. The bottom sub-figure illustrates the resulting automated agent chat from the two-agent system during program execution.

By allowing custom agents that can converse with each other, conversable agents in AutoGen serve as a useful building block. However, to develop applications where agents make meaningful progress on tasks, developers also need to be able to specify and mold these multi-agent conversations.

2.2 Conversation Programming

As a solution to the above problem, AutoGen utilizes *conversation programming*, a paradigm that considers two concepts: the first is *computation* – the actions agents take to compute their response in a multi-agent conversation. And the second is *control flow* – the sequence (or conditions) under which these computations happen. As we will show in the applications section, the ability to program these helps implement many flexible multi-agent conversation patterns. In AutoGen, these computations are conversation-centric. An agent takes actions relevant to the conversations it is involved in and its actions result in message passing for consequent conversations (unless a termination condition is satisfied). Similarly, control flow is conversation-driven – the participating agents’ decisions on which agents to send messages to and the procedure of computation are functions of the inter-agent conversation. This paradigm helps one to reason intuitively about a complex workflow as agent action taking and conversation message-passing between agents.

Figure 2 provides a simple illustration. The bottom sub-figure shows how individual agents perform their role-specific, conversation-centric computations to generate responses (e.g., via LLM inference calls and code execution). The task progresses through conversations displayed in the dialog box. The middle sub-figure demonstrates a conversation-based control flow. When the assistant receives a message, the user proxy agent typically sends the human input as a reply. If there is no input, it executes any code in the assistant’s message instead.

AutoGen features the following design patterns to facilitate conversation programming:

1. **Unified interfaces and auto-reply mechanisms for automated agent chat.** Agents in AutoGen have unified conversation interfaces for performing the corresponding conversation-centric computation, including a `send/receive` function for sending/receiving messages and a `generate_reply` function for taking actions and generating a response based on the received message. AutoGen also introduces and by default adopts an **agent auto-reply** mechanism to realize conversation-driven control: Once an agent receives a message from another agent, it automatically invokes `generate_reply` and sends the reply back to the sender unless a termination condition is satisfied. AutoGen provides built-in reply functions based on LLM inference, code or function execution, or human input. One can also register custom reply functions to customize the behavior pattern of an agent, e.g., chatting with another agent before replying to the sender agent. Under this mechanism, once the reply functions are registered, and the conversation is initialized, the conversation flow is naturally induced, and thus the agent conversation proceeds naturally without any extra control plane, i.e., a special module that controls the conversation flow. For example, with the developer code in the blue-shaded area (marked “Developer Code”) of Figure 2, one can readily trigger the conversation among the agents, and the conversation would proceed automatically, as shown in the dialog box in the grey shaded area (marked “Program Execution”) of Figure 2. The auto-reply mechanism provides a decentralized, modular, and unified way to define the workflow.
2. **Control by fusion of programming and natural language.** AutoGen allows the usage of programming and natural language in various control flow management patterns: 1) **Natural-language control via LLMs.** In AutoGen, one can control the conversation flow by prompting the LLM-backed agents with natural language. For instance, the default system message of the built-in AssistantAgent in AutoGen uses natural language to instruct the agent to fix errors and generate code again if the previous result indicates there are errors. It also guides the agent to confine the LLM output to certain structures, making it easier for other tool-backed agents to consume. For example, instructing the agent to reply with “TERMINATE” when all tasks are completed to terminate the program. More concrete examples of natural language controls can be found in Appendix C. 2) **Programming-language control.** In AutoGen, Python code can be used to specify the termination condition, human input mode, and tool execution logic, e.g., the max number of auto replies. One can also register programmed auto-reply functions to control the conversation flow with Python code, as shown in the code block identified as “Conversation-Driven Control Flow” in Figure 2. 3) **Control transition between natural and programming language.** AutoGen also supports flexible control transition between natural and programming language. One can achieve transition from code to natural-language control by invoking an LLM inference containing certain control logic in a customized reply function; or transition from natural language to code control via LLM-proposed function calls (Eleti et al., 2023).

In the conversation programming paradigm, one can realize multi-agent conversations of diverse patterns. In addition to static conversation with predefined flow, AutoGen also supports dynamic conversation flows with multiple agents. AutoGen provides two general ways to achieve this: 1) Customized `generate_reply` function: within the customized `generate_reply` function, one agent can hold the current conversation while invoking conversations with other agents depending on the content of the current message and context. 2) Function calls: In this approach, LLM decides whether or not to call a particular function depending on the conversation status. By messaging additional agents in the called functions, the LLM can drive dynamic multi-agent conversation. In addition, AutoGen supports more complex dynamic group chat via built-in GroupChatManager, which can dynamically select the next speaker and then broadcast its response to other agents. We elaborate on this feature and its application in Section 3. We provide implemented working systems to showcase all these different patterns, with some of them visualized in Figure 3.

3 Applications of AutoGen

We demonstrate six applications using AutoGen (see Figure 3) to illustrate its potential in simplifying the development of high-performance multi-agent applications. These applications are selected based on their real-world relevance (A1, A2, A4, A5, A6), problem difficulty and solving capabilities enabled by AutoGen (A1, A2, A3, A4), and innovative potential (A5, A6). Together, these criteria showcase AutoGen’s role in advancing the LLM-application landscape.